

```
In [ ]: import marimo as mo
import pandas as pd
from ortools.sat.python import cp_model
import time
```

Dados de entrada

Nós carregamos os dados de entrada de cada ficheiro de uma pasta `dados_folder` com a função `read_csv` e retornamos um dicionário agregando-os (cumprindo R8).

```
In [ ]: def carregar(dados_folder):
    # r8
    dados_dir = mo.notebook_dir() / dados_folder

    turmas = pd.read_csv(dados_dir / "turmas.csv")
    disciplinas = pd.read_csv(dados_dir / "disciplinas.csv")
    salas = pd.read_csv(dados_dir / "salas.csv")
    excecoes = pd.read_csv(dados_dir / "disponibilidade_excecoes.csv")

    # trocar NaN por string vazia
    disciplinas["sala_especial"] = disciplinas["sala_especial"].fillna("")

    return {
        "turmas": turmas,
        "disciplinas": disciplinas,
        "salas": salas,
        "excecoes": excecoes,
    }

dados1 = carregar("dados")
dados2 = carregar("dados_v2")
```

Geração de horários

1. Variáveis de Decisão

A base para o modelo é uma sequência tridimensional de uma variável x . Para cada turma (t), disciplina (d), dia da semana (dia) e período (p), temos que:

$$x_{t,d,dia,p} \in \{0, 1\}$$

Se $x == 1$, a aula ocorre naquele exato espaço e tempo.

Para facilitar o acesso à disciplina de cada professor, utilizamos um dicionário `prof_disc` que mapeia cada docente à sua lista de disciplinas.

2. Requisitos obrigatórios (R1-R8)

Impomos as restrições obrigatórias iterando nas dimensões relevantes da variável x :

- R1: usamos `add_at_most_one` iterando sobre todas as disciplinas para garantir que uma turma nunca tem mais do que uma aula no mesmo período;
- R2: a soma de todas as variáveis x de uma disciplina numa dada turma ao longo da semana é forçada a ser estritamente igual ao valor `carga_semanal`.
- R3 e R4 (Aulas diárias e blocos duplos): para disciplinas de período simples (`duplo_periodo = nao`), aplicamos `add_at_most_one` por dia, forçando as aulas a espalharem-se pela semana. para disciplinas duplas (`duplo_periodo = sim`), construímos os possíveis arranjos da mesma disciplina e fornecemos-nas ao solver através de `add_allowed_assignment`.
- R5 (Conflitos de professor): mapeamos quais disciplinas pertencem a cada professor e garantimos, via `add_at_most_one`, que o docente só leciona uma turma/disciplina por período.
- R6 (Exceções): consultamos o ficheiro de exceções e forçamos que a variável x seja igual a 0 nos períodos em que os professores não podem dar aulas.
- R7 (Capacidade das salas): para cada período, agrupamos as disciplinas relacionadas pelos seus requisitos de sala e forçamos que a soma dessas seja menor ou igual à `quantidade` global de salas desse tipo.
- R8 (Dados de entrada): ver célula anterior.

3. Função objetivo (O1)

Para evitar "buracos" no horário de cada professor, simulamos sua atividade num determinado período do dia através de uma variável u e detectamos possíveis buracos com infínimos e supremos (conjunção e disjunção), minimizando (ponto 3 da próxima secção) interrupções nos horários de cada um dos professores.

4. Construção incremental (R9)

Para cenários onde alguns dos recuross mudam ligeiramente recalcular um modelo do zero pode alterar os horários de outras turmas/professores desnecessariamente.

Para evitar isso, podemos tirar vantagem de um horário previamente calculado que é passado como parâmetro (opcional) à função e utilizá-lo para:

1. Dar dicas ao solver (`add_hint`), permitindo que a solução tenha um ponto de partida "próximo" no espaço de pesquisa;
2. Registramos possíveis diferenças (`diffs`) em variáveis;
3. Minimizamos essas diferenças e buracos no horário individual de cada professor utilizando `minimize`, fazendo com que o solver evite a deslocação de aulas ao máximo possível.

```
In [ ]: dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]
periodos = [1, 2, 3, 4, 5]
```

```

def gerar_horarios(dados, horario_prev=None):
    model = cp_model.CpModel()
    x = {}
    tl = dados["turmas"]["turma"].tolist()
    dl = dados["disciplinas"].to_dict("records")

    # professor -> [disciplina]
    prof_disc = {}
    for d in dl:
        prof = d["professor"]
        if prof not in prof_disc:
            prof_disc[prof] = []
        prof_disc[prof].append(d["disciplina"])

    # variáveis
    for t in tl:
        for d in dl:
            d_nome = d["disciplina"]
            for dia in dias:
                for p in periodos:
                    x[(t, d_nome, dia, p)] = model.new_bool_var(
                        f"x_{t}_{d_nome}_{dia}_{p}"
                    )

    # r1: sem duas aulas em simultâneo na mesma turma
    for t in tl:
        for dia in dias:
            for p in periodos:
                model.add_at_most_one(
                    x[t, d["disciplina"], dia, p] for d in dl
                )

    # [r2, r3, r4]: restrições de disciplinas
    for t in tl:
        for d in dl:
            d_nome = d["disciplina"]
            carga = d["carga_semanal"]
            duplo = d["duplo_periodo"] == "sim"

            # r2: cada disciplina cumpre sua exata carga semanal
            model.add(
                sum(
                    x[(t, d_nome, dia, p)]
                    for dia in dias
                    for p in periodos
                )
                == carga
            )

            for dia in dias:
                vars_dia = [x[(t, d_nome, dia, p)] for p in periodos]

            # r3: se não for dupla, disciplina só pode ter uma aula por
            dia numa turma
            if not duplo:

```

```

        model.add_at_most_one(vars_dia)

        # r4: se for dupla, a disciplina tem que ser dada em blocos
consecutivos
    else:
        validos = []
        validos.append(
            [0] * len(periodos)
        ) # disciplina pode não ocorrer no dia

        for i in range(len(periodos) - 1):
            # ocorrências da disciplina (no mesmo dia e turma)
pode ser e.g. [0, 0, 1, 1, 0]
            curr = [0] * len(periodos)
            curr[i] = 1
            curr[i + 1] = 1
            validos.append(curr)

        model.add_allowed_assignments(vars_dia, validos)

# r5: professor não dá duas aulas em simultâneo
for prof, d_nomes in prof_disc.items():
    for dia in dias:
        for p in periodos:
            model.add_at_most_one(
                x[(t, d_nome, dia, p)]
                for t in tl
                for d_nome in d_nomes
            )

# r6: professor só dá aulas se disponível
for r in dados["excecoes"].itertuples(index=False):
    prof, dia, p = r.professor, r.dia, r.periodo
    if prof in prof_disc:
        for d_nome in prof_disc[prof]:
            for t in tl:
                model.add(x[(t, d_nome, dia, p)] == 0)

# r7: requisitos de salas
caps = {} # tipo da sala -> capacidade
for r in dados["salas"].itertuples():
    tipo, quant = r.tipo, r.quantidade
    caps[r.sala if tipo == "especial" else "normal"] = quant

for dia in dias:
    for p in periodos:
        for req, cap in caps.items():
            # construímos uma lista de disciplinas "relacionadas" (com
os mesmos requisitos de sala) para cada disciplina simultânea
            if req == "normal":
                disc_rel = [
                    d["disciplina"]
                    for d in dl
                    if not d["sala_especial"]
                ] # salas normais são vazias
            else:

```

```

        disc_rel = [
            d["disciplina"]
            for d in dl
            if d["sala_especial"] == req
        ]

        # soma de todas as disciplinas relacionadas deve ser menor
        # que a capacidade total
        if disc_rel:
            model.add(
                sum(
                    x[(t, d_nome, dia, p)]
                    for t in tl
                    for d_nome in disc_rel
                )
                <= cap
            )

        # o1: começamos por coletar os buracos nos horários de cada professor
        # numa nova variável
        buracos = []
        for prof, d_nomes in prof_disc.items():
            for dia in dias:
                u = {}
                for p in periodos:
                    u[p] = model.new_bool_var(
                        f"u_{prof}_{dia}_{p}"
                    ) # u[p] é verdadeiro se o professor trabalha em período p
                    model.add_max_equality(
                        u[p],
                        [
                            x[(t, d_nome, dia, p)]
                            for t in tl
                            for d_nome in d_nomes
                        ],
                    )

                # só pode haver um buraco nos períodos 2, 3 e 4
                for p in range(2, 5):
                    a = model.new_bool_var(f"antes_{prof}_{dia}_{p}")
                    d = model.new_bool_var(f"depois_{prof}_{dia}_{p}")

                    # a (d) é verdadeiro se há aula antes (depois) de a (d)
                    model.add_max_equality(a, [u[i] for i in range(1, p)])
                    model.add_max_equality(d, [u[i] for i in range(p + 1, 6)])
                    b = model.new_bool_var(f"buraco_{prof}_{dia}_{p}")

                    # b é verdadeiro se há aulas antes e depois, mas não no
                    # momento
                    model.add_min_equality(b, [a, d, u[p].Not()])
                    buracos.append(b)

        # r9: construção incremental a partir de um horário prévio como base
        if horario_prev is not None:
            diffs = []
            for (t, d_nome, dia, p), var in x.items():

```

```

        if (t, d_nome, dia, p) in horario_prev:
            var_prev = horario_prev[(t, d_nome, dia, p)]
            model.add_hint(var, var_prev) # dicas para o solver
            diffs.append(
                var.Not() if var_prev else var
            ) # anotamos possiveis diferenças nas variáveis
        else:
            diffs.append(var) # nova aula que não existia antes

    # amplificamos as mudanças (para maior minimização e somamos aos buracos)
    model.minimize(100 * sum(diffs) + sum(buracos))
else:
    model.minimize(sum(buracos))

solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 30.0

status = solver.solve(model)

if status in [cp_model.OPTIMAL, cp_model.FEASIBLE]:
    return (
        {k: solver.value(v) for k, v in x.items()},
        status,
        solver.objective_value,
    )

return None, status, None

```

Fluxo e análise

Começamos por gerar `h0` -- o horário inicial calculado a partir do conjunto presente na pasta `dados/`.

```

In [ ]: start_h0 = time.time()
        h0, status0, obj0 = gerar_horarios(dados1)
        time_h0 = time.time() - start_h0

```

Geramos agora, a partir de `dados_v2/`, `h1` e `h1_incremental`, correspondendo respectivamente a um horário gerado do zero, e um horário que utiliza o horário `h0` como base.

```

In [ ]: start_h1 = time.time()
        h1, status1, obj1 = gerar_horarios(dados2)
        time_h1 = time.time() - start_h1

        start_h1_incremental = time.time()
        h1_incremental, status1_incremental, obj1_incremental = gerar_horarios(
            dados2, horario_prev=h0
        )
        time_h1_incremental = time.time() - start_h1_incremental

```

Por fim, calculamos alterações entre os horários e elaboramos uma análise comparativa entre estes utilizando como métrica o tempo de execução total, e as alterações totais entre as aulas.

```
In [ ]: aulas_h0 = {k for k, v in h0.items() if v == 1}
aulas_h1 = {k for k, v in h1.items() if v == 1}
aulas_h1_incremental = {k for k, v in h1_incremental.items() if v == 1}

# alterações de h1 (gerado do zero) em comparação a h0
# dividimos por 2 pois cada discrepância gera duas diferenças
alteracoes = len(aulas_h0.symmetric_difference(aulas_h1)) // 2

# alterações de h1 (gerado incrementalmente) em comparação a h0
alteracoes_incremental = (
    len(aulas_h0.symmetric_difference(aulas_h1_incremental)) // 2
)

def formatar(h, t):
    if not h:
        return pd.DataFrame()

    aulas = [k for k, v in h.items() if v == 1 and k[0] == t]
    df = pd.DataFrame(
        aulas, columns=["Turma", "Disciplina", "Dia", "Periodo"]
    )

    ordem_dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]

    df_pivot = df.pivot(
        index="Periodo", columns="Dia", values="Disciplina"
    ).fillna("-")

    # ordem cronológica
    colunas_presentes = [
        dia for dia in ordem_dias if dia in df_pivot.columns
    ]
    return df_pivot[colunas_presentes]

def gerar_abas(h):
    if not h:
        return mo.md("horário não gerado")

    # todas as turmas únicas presentes nas chaves deste horário específico
    turmas_h = sorted(list(set(k[0] for k in h.keys())))

    abas = {}
    for t in turmas_h:
        df_t = formatar(h, t)
        abas[f"Turma {t}"] = mo.as_html(df_t)

    return mo.ui.tabs(abas) if abas else mo.md("nenhuma turma encontrada")

interface = mo.ui.tabs(
    {
```

```

        "H0 (original)": gerar_abas(h0),
        "H1 (do zero)": gerar_abas(h1),
        "H1 (incremental)": gerar_abas(h1_incremental),
    }
)

mo.md(f"""
### Análise da construção incremental

| Horário | Tempo | Aulas alteradas (vs H0) |
| :--- | :--- | :---
| **H0** | `{time_h0}` | N/A |
| **H1 (do zero)** | `{time_h1}` | `{alteracoes}` aulas alteradas |
| **H1 (incremental)** | `{time_h1_incremental}` |
  `{alteracoes_incremental}` aulas alteradas |

Temos uma diferença de tempo entre H1 (do zero) e H1 (incremental) de
  `{time_h1 - time_h1_incremental}` segundos.

### Visualização interativa de horários

{interface}
""")

```

Análise da construção incremental

Horário	Tempo	Aulas alteradas (vs H0)
H0	0.0497431755065918	N/A
H1 (do zero)	0.0480349063873291	31 aulas alteradas
H1 (incremental)	0.039989471435546875	0 aulas alteradas

Temos uma diferença de tempo entre H1 (do zero) e H1 (incremental) de 0.008045434951782227 segundos.

Visualização interativa de horários